

LangChain Components

LangChain provides a modular ecosystem for building LLM-based applications. At its core, it has **six major components**:

1. **Models**
2. **Prompts**
3. **Chains**
4. **Memory**
5. **Indexes**
6. **Agents**

Understanding these six components gives you clarity on how LangChain applications are structured.

In this section, we explore the **Model Component** in detail.

1. Model Component

◆ What Are Models in LangChain?

Models are the **core interfaces** through which LangChain interacts with various AI models.

These include both:

- **Language Models (LLMs)** – generate text
- **Embedding Models** – generate vector embeddings of text

LangChain standardizes communication with different model providers so developers don't need to learn multiple APIs.

1.1 Why Models Were Needed – The Background

Before LLMs

Traditionally, building NLP applications like chatbots was difficult because:

- Understanding user queries (NLU) was hard.
- Generating context-aware responses was even harder.

After LLMs

LLMs solved both problems together:

- They understand natural language.

- They generate fluent, contextual responses.

But LLMs brought new challenges:

- LLM files are huge (often > 100 GB).
- Running them locally is expensive.
- Every provider exposes *different APIs*.

API Problem

Different LLM providers = different code styles

Example:

- One syntax for one provider
- Another syntax for another provider

Switching models → You had to rewrite code.

1.2 How LangChain Solves This

LangChain provides a **standardized interface** to communicate with any LLM provider. You change only **1–2 lines of code** when switching between models.

Example:

- If you move from Provider A to Provider B
- The calling pattern & result handling stay the same
- Only the model name/provider import changes

This makes your code:

- Cleaner
 - Scalable
 - Vendor-independent
-

1.3 Types of Models in LangChain

A. Language Models

These follow **text-in** → **text-out** behavior.

Used for:

- Chatbots
- Agents
- Summarization
- Content generation
- RAG responses

B. Embedding Models

These follow **text-in** → **vector-out** behavior.

Used for:

- Semantic Search
- RAG
- Similarity Matching
- Clustering & Indexing

Embedding models are essential for building search-based AI pipelines.

1.4 Supported Providers (Common Examples)

LangChain supports many model providers for both chat and embeddings, such as:

Chat/Language Models

- OpenAI
- Anthropic
- Google (Vertex AI)
- AWS Bedrock
- Mistral
- HuggingFace
- Many more...

Each provider supports capabilities like:

- Tool calling
- Structured output
- JSON mode
- Multimodal inputs (depending on model)

Embedding Models

- OpenAI
 - Mistral
 - HuggingFace
 - IBM models
 - Llama-based embeddings
-

1.5 Summary — What the Model Component Does

Aspect	Explanation
Purpose	Standard interface to communicate with any AI model
Solves	Different providers having different API styles
Benefits	Easy switching between models, minimal code changes
Types	Language Models & Embedding Models
Importance	One of the most essential components of LangChain

✓ Key Takeaway

LangChain's Model Component standardizes how you talk to LLMs, making it easy to switch between providers without rewriting your entire codebase.

2. LangChain Components → Prompts

◆ What Are Prompts?

- In LLMs, whatever *input* you send to the model is called a **prompt**.
- Example:
 - If you ask ChatGPT: "**What is CampusX?**" → this text string is your **prompt**.

◆ Why Are Prompts Important?

- LLM outputs are **highly sensitive** to how prompts are written.
- Even a small wording change can drastically change the answer.

Example:

- *Explain Linear Regression in academic tone*
- *Explain Linear Regression in fun tone*
 - Only one word changes, yet the entire output style changes.

Because of this, a new field has emerged:

◆ Prompt Engineering

- Critical skill in the LLM era
- Many companies hire **Prompt Engineers**
- Despite memes on social media, it's *very important* for real-world LLM applications

2.1 Prompt Component in LangChain

LangChain recognizes that:

- Prompts are essential
- Prompts need to be **dynamic, powerful, flexible, reusable**

Therefore, LangChain provides a **dedicated Prompt component** that helps you design:

- Dynamic prompts
 - Role-based prompts
 - Few-shot prompts
 - Structured prompts
 - Templates with placeholders
-

3. Types of Prompts You Can Create in LangChain

✓ 1. Dynamic & Reusable Prompts

Sometimes you don't know:

- What topic the user will ask
- What tone the answer should be in

So you make a flexible template:

Summarize {topic} in {tone}

- User 1: "Cricket in fun tone"
→ Filled prompt becomes: *Summarize cricket in fun tone*
- User 2: "Biology in serious tone"
→ *Summarize biology in serious tone*

➡ Helps create scalable, reusable prompt systems

✓ 2. Role-Based Prompts

You can guide the LLM to behave like a specific expert.

System Prompt Template:

Hi, you are an experienced {profession} .

User Prompt Template:

Tell me about {topic} .

Examples:

- Profession → *Doctor*
Topic → *Viral fever*

- Profession → *Engineer*
Topic → *Bridge design*

→ Very useful for chatbots, domain-specific advisors, automation

✓ 3. Few-Shot Prompts

Few-shot prompts = You first show the LLM **examples**, then ask your real question.

Used heavily in:

- Customer support bots
- Classification tasks
- Pattern-learning tasks
- Content tagging

Examples shown to LLM:

User Message	Category
I was charged twice for my subscription	Billing Issue
The app crashes every time I log in	Technical Problem
Can you explain how to upgrade my plan?	General Inquiry

Then you ask:

Classify the following ticket:

"My screen is showing error code 500."

LLM learns from examples → responds intelligently.

This template is very powerful for controlled, reliable outputs.

Final Summary (Short & Clean)

- Prompts = Inputs to LLM
 - Prompts influence output heavily
 - LangChain provides a robust system to create:
 - Dynamic prompts
 - Role-based prompts
 - Few-shot prompts
 - Prompt Engineering is an essential skill for building LLM applications
-

3. LangChain Components → Chains

◆ What Are Chains?

Chains are one of the **most important components** of LangChain — in fact, the name *LangChain* comes from this idea.

A **Chain** = A pipeline of multiple steps

Where:

- Output of Step 1 → Automatically becomes input of Step 2
- No manual data passing required
- LangChain handles the entire flow behind the scenes

This allows you to convert any LLM application into a clean, modular pipeline.

3.1 Why Chains Are Needed?

If you don't use chains, you must:

1. Take user input manually
2. Call LLM-1
3. Collect output manually
4. Pass it into LLM-2
5. Collect output again
6. Continue repeating this...

This becomes **complex, messy**, and error-prone.

Chains solve this by **automating the entire pipeline**.

3.2 Example: Sequential Chain

Task:

User gives a 1000-word English text → You must give a **Hindi summary in <100 words**

Pipeline:

1. Step 1 → Translate English to Hindi (LLM-1)
2. Step 2 → Summarize the Hindi text (LLM-2)

Without Chains

→ You manually pass outputs between steps.

With Chains

→ You simply call the chain once:

Input → Chain → Final Output

Behind the scenes, the chain automatically:

- Sends input to LLM-1

- Takes its output
- Sends to LLM-2
- Returns the final summary

➡ This is the beauty of **Sequential Chains**.

3.3 Other Types of Chains in LangChain

✓ 1. Sequential Chains

One step after another.

Ideal for:

- Translate → Summarize
 - Extract → Validate → Format
 - Retrieve → Answer
-

✓ 2. Parallel Chains

Used when multiple LLMs must run **at the same time**.

Example Scenario: User provides: "9/11 incident"

You want:

- LLM-1 → Historical overview
- LLM-2 → Political analysis
- LLM-3 → Combine both into a final report

Pipeline: Input → **LLM1 (Report A)**

→ **LLM2 (Report B)**

→ **LLM3 combines A + B**

→ **Final Output**

This is a **Parallel Chain**.

Useful for:

- Multi-expert LLM systems
 - Answer cross-verification
 - Ensemble reasoning
 - Multi-agent outputs
-

✓ 3. Conditional Chains

Based on logic or conditions, the chain takes different paths.

Example: Customer Feedback Agent

1. User gives feedback
2. LLM analyzes sentiment
3. If positive → Reply with "Thank you!"
4. If negative → Automatically send an email to support team

If sentiment = good → path A

If sentiment = bad → path B

Great for:

- Automated workflows
 - AI agents
 - Support automation
 - Decision-based pipelines
-

3.4 Why Chains Are So Powerful?

- They reduce manual code
- They reduce errors
- They create clean, modular pipelines
- They support sequential, parallel, and conditional flows
- They allow building **complex LLM workflows quickly**

LangChain's design makes it extremely easy to build:

- LLM pipelines
 - Multi-step reasoning flows
 - Agent architectures
 - Data processing workflows
 - Enterprise-level applications
-

Final Takeaway

Chains = The core mechanism that turns your LLM tasks into structured pipelines
→ and LangChain automates everything behind the scenes.

In []:

4. LangChain Components → Indexes

◆ What Are Indexes?

Indexes allow your LLM application to **connect with external knowledge sources** such as:

- PDFs

- Websites
- Databases
- Any private/internal documents

With Indexes, your model gains access to information **outside** its original training data.

4.1 Why Indexes Are Needed?

LLMs like ChatGPT are trained on public internet data.

So they can answer:

- "Who is the Prime Minister of India?"

But they **cannot** answer:

- "What is the leave policy of my company XYZ?"
- "What is the notice period in my organization?"

Because this data is **private** and not part of model training.

Solution:

Connect your LLM to your **own documents** using Indexes.

This allows the LLM to answer questions based on:

- Your organization's rulebook
 - Your PDFs
 - Your internal databases
-

4.2 Components of Indexes

Indexes consist of **four sub-components**:

✓ 1. Document Loader

Loads data from any source:

- PDFs
- Text files
- GDrive
- S3 bucket
- Websites
- SQL/NoSQL databases

Purpose: Bring your external data into your application.

✓ 2. Text Splitter

Large documents (like a 1000-page policy book) cannot be embedded directly.

So Text Splitters break them into smaller chunks:

- By pages
- By paragraphs
- By sections

Example:

1000 pages → 1000 chunks

Purpose: Prepare text for semantic search.

✓ 3. Vector Store

For semantic search, we convert each chunk to embeddings (vectors).

Example:

- 1000 chunks → 1000 embedding vectors
- These vectors are stored in a special database called **Vector Store**

Popular Vector Stores include:

- Pinecone
- Chroma
- Weaviate
- FAISS
- Milvus

Purpose: Store and retrieve vectors efficiently.

✓ 4. Retriever

When the user asks a question (e.g., "What is the leave policy?"):

1. Retriever converts the query into an embedding
2. Performs semantic search in the vector store
3. Retrieves the most relevant chunks
4. Sends these retrieved chunks + user query to the LLM
5. LLM generates a final answer

Purpose: Find relevant knowledge from your data.

4.3 How Indexes Work Together (Full Flow)

Let's assume:

- A company XYZ has a 1000-page rules PDF
- You want the LLM to answer questions using that PDF

Process:**1. Document Loader:**

- Load the 1000-page PDF from GDrive or S3

2. Text Splitter:

- Split into 1000 chunks (1 chunk per page)

3. Vector Store:

- Create embeddings for each chunk
- Store embeddings in a vector database

4. Retriever:

- User asks: *"What is the leave policy of XYZ?"*
- Retriever creates query embedding
- Searches vector store
- Retrieves relevant pages
- Sends to LLM along with question

5. LLM:

- Reads retrieved context
- Generates correct answer
- Even though it was not trained on this private data

→ **This is how LLMs can use external company-specific knowledge.**

4.4 Why Indexes Are Powerful?

- They allow LLMs to answer **enterprise/private queries**
 - You can connect any data source
 - They enable powerful semantic search
 - No need to fine-tune models
 - Perfect for building RAG (Retrieval-Augmented Generation) applications
 - LangChain makes the entire pipeline extremely easy
-

Final Takeaway

Indexes = The mechanism in LangChain that connects your LLM to external knowledge.
They consist of 4 parts:

1. Document Loaders
2. Text Splitters
3. Vector Stores

4. Retrievers

All four work together to enable **semantic search + context-aware answers** using your own documents.

Let me know when you want the next LangChain component (Memory, Models, or Agents) in Jupyter Markdown.

6. LangChain Component: Memory

Introduction

LLM API calls are **stateless**, meaning each request is processed independently without remembering earlier messages. This becomes a big limitation when building chatbots or conversational applications, because the model quickly “forgets” the context of the conversation.

Example:

- **User:** Who is Narendra Modi?
- **Model:** Narendra Modi is the current Prime Minister of India.
- **User:** How old is he?

Without memory, the model cannot link “he” to “Narendra Modi”, and returns a generic response.

This makes the chatbot frustrating to use.

Why Memory Is Needed

To maintain meaningful conversations, the LLM must remember:

- What was previously asked
- What it replied
- The context of the topic
- User preferences (optional)

LangChain solves this problem through its **Memory** module.

Types of Memory in LangChain

LangChain provides multiple memory mechanisms depending on the use-case.

1. ConversationBufferMemory

- Stores **entire conversation history**.
- Sends full history back to the model with every new API call.
- Simple and most commonly used.

- **Limitation:**
If the conversation becomes long, the token cost increases heavily.
-

2. ConversationBufferWindowMemory

- Instead of storing the full history, it stores the **last N interactions**.
 - You can define the window size.
Example: Keep only the last 100 messages.
 - Balances context + cost efficiency.
-

3. ConversationSummaryMemory

- Summarizes the entire chat history and stores the **summary** instead of raw text.
 - In every API call, the summary is passed to the LLM.
 - Saves tokens and reduces cost.
 - Useful for very long sessions.
-

4. Custom Memory

- Used for advanced or specialized cases.
 - You choose what to store:
 - User preferences
 - Specific facts
 - Session-specific structured data
 - Often used in personalized assistants or recommendation-based systems.
-

Why Memory Matters (Simple Explanation)

Memory lets your LLM-based application:

- Maintain conversation flow
- Understand pronouns like "he", "she", "that policy", etc.
- Provide personalized responses
- Avoid breaking context
- Give a human-like chat experience

Without memory, every message feels like a fresh conversation.

With memory, the model behaves like a consistent conversational agent.

Summary

LangChain Memory solves the stateless nature of LLMs by adding context-awareness. Depending on your needs, you can choose buffer memory, window memory, summary

memory, or custom memory.

In the upcoming sections of your notes, you will explore **practical implementations** of each memory type in LangChain.

7. LangChain Component: Agents

Introduction

Agents are one of the most powerful components in LangChain.

They allow you to build **AI agents**—systems that can *understand, reason, and take actions* using external tools.

In the last few months, AI agents have become a major trend because they move beyond simple chatbots.

They do not just talk—they can **perform tasks**.

What Exactly Is an AI Agent?

LLMs have two core strengths:

1. **NLU (Natural Language Understanding)** → They understand user queries.
2. **Text Generation** → They generate meaningful responses.

This is why the first obvious use-case of LLMs was **chatbots**, including ChatGPT itself.

But developers asked a bigger question:

If a chatbot can understand me,
can it *also* do tasks for me?

This thought gave rise to **AI Agents**.

Difference Between a Chatbot and an AI Agent

Chatbot	AI Agent
Only understands and replies	Understands + reasons + takes actions
Limited to conversation	Can use tools & APIs
Cannot perform tasks	Can perform tasks (book flights, fetch data, calculate, etc.)

An AI agent is basically a *chatbot with superpowers*.

How an AI Agent Works

An AI agent has two special abilities that a normal chatbot does not:

1. Reasoning Ability

The agent can break down a problem step-by-step.

Common technique: **Chain-of-Thought reasoning**.

It looks at the user query, decides what steps to take, and plans how to solve the problem.

2. Access to Tools

Agents can use external tools such as:

- APIs (weather, flights, stock prices, etc.)
- Calculators
- Databases
- Custom functions

Because of these tools, the agent can *take actions*, not just reply.



Example: How an Agent Responds

Imagine we built an agent and gave it two tools:

- **Calculator**
- **Weather API**

User query:

Multiply today's temperature of Delhi by 3.

Step-by-step reasoning by the agent:

1. Understand the query → Need today's temperature of Delhi.
2. Check available tools → Weather API is available.
3. Call the Weather API with "Delhi" → Returns **25°C**.
4. Now compute 25×3 → Needs calculator tool.
5. Call calculator → Output **75**.
6. Return "75" as the final answer.

This is exactly how real AI agents operate:

- Think
 - Select tool
 - Use tool
 - Continue reasoning
 - Produce final result
-

Why AI Agents Matter

Agents represent the **next evolution** after chatbots.

They allow AI systems to:

- Fetch live information
- Search the web
- Book tickets
- Analyze documents
- Run workflows
- Interact with databases
- Perform multi-step tasks

This is why most major AI companies are focusing heavily on agent-based systems.

Agents in LangChain

LangChain makes building AI agents extremely easy:

- You define the tools
- You define the LLM
- LangChain automatically handles reasoning + tool selection
- You get a working agent in very few lines of code

In the upcoming sections of your notebook, you will learn:

- How to build agents
 - How tools are defined
 - How workflows are created
 - Real agent projects step-by-step
-

Conclusion

Agents are the final major component of LangChain.

They combine **reasoning + tools** to build AI systems that can take real actions, not just chat.

With this, the introduction to all LangChain components is complete.

Next, we will study each component in detail with practical use-cases.

In []: